

¿Qué es un Agente LLM?

Módulo 10 · Sistemas Agénticos y Multi-Agente

Los cuatro componentes del agente, ciclo perceive-reason-act, agentes vs pipelines y principios de producción.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

¿Qué es un Agente LLM?

Un LLM estándar es reactivo: recibe una entrada, produce una salida, y termina. Un agente LLM es activo: recibe un objetivo, planifica la secuencia de acciones necesaria para alcanzarlo, ejecuta esas acciones (llamando a herramientas externas, consultando bases de datos, escribiendo y ejecutando código), observa los resultados, ajusta su plan, y continúa hasta completar el objetivo. La diferencia es la que existe entre una calculadora y un piloto: uno computa, el otro navega.

En 2025, la adopción de sistemas agénticos en empresa ha aumentado dramáticamente. El 72% de los proyectos de IA empresarial ahora involucran arquitecturas multi-agente, frente al 23% en 2024 (según datos de orquestación de 2025). El sector de agentes de IA se proyecta desde 5.4 billones de dólares en 2024 hasta 50.3 billones en 2030. Anthropic, OpenAI, Google y Microsoft han publicado frameworks de agentes en 2025: Claude SDK con sub-agentes, OpenAI Agents SDK (marzo 2025), Google ADK (abril 2025), y Microsoft Agent Framework (octubre 2025). La arquitectura agéntica se está convirtiendo en el paradigma dominante para la automatización empresarial compleja.

Los cuatro componentes de un agente LLM

1. Percepción: leer el entorno

El agente percibe su entorno a través de los inputs que recibe: el objetivo del usuario, el estado actual del proceso, los resultados de las herramientas ejecutadas, y cualquier feedback del entorno. A diferencia de un LLM estándar que solo recibe un prompt, el agente mantiene un estado acumulativo que incluye toda la historia de acciones y observaciones del proceso actual. Los agentes multimodales pueden percibir no solo texto sino también imágenes, audio, y datos estructurados.

2. Razonamiento y planificación: decidir qué hacer

El LLM actúa como el "cerebro" del agente: dada la percepción del estado actual y el objetivo, razona sobre qué acción es más apropiada para avanzar hacia el objetivo. Este razonamiento puede ser explícito (CoT, ReAct: el agente genera pensamientos visibles antes de actuar) o implícito (en modelos de razonamiento como o3 que piensan internamente). El agente no sigue un flujo predeterminado: genera su propio plan de acción dinámicamente basándose en el estado actual.

3. Acción y uso de herramientas: hacer cosas en el mundo

El agente ejecuta acciones a través de herramientas: funciones Python que puede llamar con los argumentos correctos. Las herramientas típicas incluyen: búsqueda web, ejecución de código, consulta a bases de datos, envío de emails, llamadas a APIs externas, lectura/escritura de archivos, y comunicación con otros agentes. La calidad del set de herramientas disponibles determina en gran medida qué objetivos puede alcanzar el agente.

4. Memoria: mantener el contexto a lo largo del tiempo

La memoria del agente es lo que le permite mantener coherencia a lo largo de tareas multi-paso y aprender de experiencias pasadas. Los tipos de memoria son: la ventana de contexto actual (memoria de trabajo), el historial de la sesión actual (memoria episódica a corto plazo), las memorias persistentes entre sesiones (memoria a largo plazo en bases de datos), y el conocimiento del dominio (memoria semántica, vía RAG). La gestión de la memoria es uno de los desafíos técnicos más importantes de los sistemas agénticos.

SECCIÓN 3 · CÓMO FUNCIONA

El ciclo de vida de un agente: perceive-reason-act

El ciclo básico de un agente es: recibir el objetivo/estado → razonar sobre qué hacer → seleccionar y ejecutar una acción (tool call) → observar el resultado → actualizar el estado → decidir si continuar o terminar. Este ciclo se repite hasta que el agente determina que ha completado el objetivo o ha alcanzado el límite máximo de iteraciones. La implementación más común en 2025 usa function calling (OpenAI/Anthropic API) donde el modelo decide dinámicamente qué función llamar basándose en su razonamiento.

Los límites de las iteraciones son críticos en producción: sin un `max_iterations`, un agente puede entrar en bucles infinitos o consumir cientos de llamadas al API antes de detectar que no puede completar la tarea. El límite estándar es 10-25 iteraciones para la mayoría de las tareas de producción, con logging de cada iteración para debugging.

Agentes vs pipelines: cuándo usar cada uno

Un pipeline LLM es una secuencia predeterminada de pasos: recuperar → resumir → clasificar. El flujo es fijo y determinista. Un agente LLM determina su propio flujo basándose en el estado actual: puede hacer más recuperaciones si la primera no fue suficiente, puede decidir no clasificar si el texto no requiere clasificación, puede detectar que necesita información adicional antes de continuar. Los pipelines son más eficientes y predecibles; los agentes son más flexibles pero más costosos y difíciles de depurar. Anthropic recomienda: "Si los pipelines son suficientes para el caso de uso, úsalos."

SECCIÓN 4 · EJEMPLOS REALES

- Agente de investigación de due diligence: objetivo "analiza la empresa X para una posible adquisición". El agente decide: buscar en web noticias recientes → buscar en base de datos de empresas (Crunchbase, LinkedIn) → recuperar informes financieros → analizar los resultados → generar informe. La secuencia exacta depende de lo que encuentra en cada paso, no de un flujo predeterminado.
- GitHub Copilot Workspace (2024): un agente que recibe un issue de GitHub, planifica los cambios necesarios, edita los archivos relevantes, ejecuta los tests, corrige errores si fallan, y propone un PR. OSWorld benchmark (junio 2025): los modelos más capaces alcanzan 42.9% de tasa de completación de tareas en sistemas operativos vs 72.36% para humanos.
- Agente de atención al cliente autónomo (Zendesk + Anthropic): el agente puede consultar el CRM, verificar el historial de compras, procesar devoluciones, y escalar a un agente

humano si la situación lo requiere. La decisión de cuándo escalar es del agente, no de un árbol de decisión predeterminado.

- Devin (Cognition AI): agente de software engineering que puede leer documentación, ejecutar comandos de terminal, editar código, ejecutar tests, y corregir errores iterativamente. Primera demostración pública de un agente capaz de resolver issues de repositorios reales de GitHub con mínima supervisión humana.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar un agente cuando un pipeline sería suficiente. Los agentes son más costosos (más llamadas al LLM), más difíciles de depurar (el flujo no es determinista), y más propensos a fallos en cascada. Si el flujo de trabajo es predecible y los pasos son bien conocidos, un pipeline predeterminado es más fiable y económico.

Error 2: Dar al agente acceso a herramientas de alto impacto sin human-in-the-loop. Un agente con acceso a enviar emails, modificar bases de datos de producción, o hacer transferencias bancarias puede hacer daño irreversible si falla. El principio de mínimo privilegio y los checkpoints de confirmación humana son imprescindibles para herramientas destructivas o de alto impacto.

Error 3: No limitar las iteraciones. Sin `max_iterations`, un agente puede entrar en un bucle de 50+ llamadas al API intentando resolver una tarea que no puede completar. Define siempre un límite máximo de iteraciones y un comportamiento de fallback cuando se alcanza.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para construir un agente LLM básico con la API de Anthropic: define las herramientas disponibles como funciones Python con sus schemas JSON. Implementa el bucle de agente: llama al modelo con las herramientas disponibles, si el modelo devuelve un `tool_use` block ejecuta la herramienta y añade el resultado al historial, repite hasta que el modelo devuelve un `text` block sin `tool_use` (tarea completada) o se alcanza `max_iterations`. El logging de cada iteración (`thought`, `tool_call`, `observation`) es imprescindible para debugging.

El principio de "start simple, scale smart": empieza con un agente de herramienta única (un agente que puede hacer una cosa bien) antes de añadir complejidad. Valida el comportamiento con casos de prueba simples antes de desplegar en producción. Añade herramientas gradualmente, evaluando el comportamiento después de cada adición. La complejidad del agente debe añadirse solo cuando hay evidencia clara de que es necesaria.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M6-T6 (ReAct): el patrón Thought/Action/Observation de ReAct es la base del ciclo de vida del agente.
- M10-T2 (Arquitectura del agente): profundiza en cada componente arquitectónico del agente.
- M10-T5 (Tool use): las herramientas son el mecanismo de acción del agente sobre el mundo.

Un agente LLM tiene cuatro componentes: percepción (leer el entorno y el estado), razonamiento/planificación (el LLM decide qué hacer), acción/herramientas (ejecutar funciones externas), y memoria (mantener contexto). El ciclo es: percibir → razonar → actuar → observar → repetir. Los agentes son apropiados cuando el flujo de trabajo no es determinístico y el agente necesita adaptar su plan basándose en los resultados intermedios. Para flujos predecibles, usa pipelines. Principios de producción: `max_iterations` siempre definido, mínimo privilegio en herramientas, human-in-the-loop para acciones destructivas, logging de cada iteración.